

GPUMODE Lecture Study Notes: Lecture 99: Distributed ML on Consumer Devices

Core Problem the Lecture Addresses

The lecture studies how to move large language model inference and even limited training away from traditional datacenter GPU clusters and onto **consumer or prosumer devices**, especially Apple Silicon machines such as Mac Studios. The central question is:

Can we run large-scale ML workloads using many consumer devices connected together?

The answer is not simply yes or no. The lecture argues that consumer devices have a very different hardware profile from NVIDIA datacenter GPUs. Apple Silicon, in particular, has relatively weak peak floating-point throughput compared with H100 or H200 GPUs, but it can offer unusually large unified memory capacity at a much lower dollar-per-gigabyte cost. That makes it attractive for workloads where **model capacity and memory bandwidth** matter more than raw compute.

The key bottleneck is therefore a distributed systems problem:

Large model $\implies$ too much memory for one device $\implies$ split the model across devices.

Once the model is split, the next bottleneck becomes communication:

Distributed inference time = local compute time + inter-device communication time + synchronization overhead

On consumer hardware, the interconnect is not NVLink. Instead, the system may use Ethernet or Thunderbolt. This means the best parallelism strategy can differ sharply from the strategy one would choose on an NVIDIA server.

The lecture focuses on four big ideas:

1. Why one might want on-device or on-premise LLMs.
2. Why Apple Silicon is unusually interesting for large-memory inference.
3. How Apple MLX differs from PyTorch and why lazy execution matters.
4. How pipeline parallelism, tensor parallelism, heterogeneous prefill/decode, and experimental training work on consumer-device clusters.

The core mental shift is that consumer-device distributed ML is not just cheaper datacenter ML. It is a different regime where memory capacity, local privacy, physical topology, communication latency, and framework semantics dominate the design.

Required Background

Large Language Model Inference

Autoregressive LLM inference usually has two phases:

1. **Prefill**: Process the entire prompt and build the key-value cache.
2. **Decode**: Generate one new token at a time using the existing key-value cache.

For a transformer with sequence length S , hidden dimension d , and number of layers L , the prefill phase performs large matrix operations over the prompt:

$$\text{Prefill work} \sim O(LSd^2) + O(LS^2d).$$

The decode phase, by contrast, processes one token at a time:

$$\text{Decode work per token} \sim O(Ld^2) + O(LSd).$$

The decode phase often has lower arithmetic intensity because each generated token requires reading a large amount of model weight data and key-value cache data for relatively little computation.

Arithmetic Intensity and the Roofline Model

A central performance concept in the lecture is **arithmetic intensity**:

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes moved}}.$$

The roofline model says that achievable throughput is bounded by both compute throughput and memory bandwidth:

$$P_{\text{achieved}} \leq \min(P_{\text{peak}}, B_{\text{mem}} \cdot I),$$

where:

- P_{achieved} is achieved performance in FLOPs per second.
- P_{peak} is the hardware peak compute throughput.
- B_{mem} is memory bandwidth in bytes per second.
- I is arithmetic intensity in FLOPs per byte.

The transition point between memory-bound and compute-bound behavior is:

$$I_{\text{critical}} = \frac{P_{\text{peak}}}{B_{\text{mem}}}.$$

If $I < I_{\text{critical}}$, the workload is memory-bound. If $I > I_{\text{critical}}$, the workload is compute-bound.

Memory-Bound Versus Compute-Bound Inference

The lecture emphasizes that LLM decode is often memory-bound. Each token requires streaming through large model weights. For a rough model with N parameters and b bytes per parameter, the bytes read per generated token are approximately:

$$\text{Bytes per token} \approx Nb.$$

If a model has 670×10^9 parameters and uses approximately 1 byte per parameter under 8-bit quantization, then the model weights alone require roughly:

$$670 \times 10^9 \text{ bytes} \approx 670 \text{ GB}.$$

This explains why memory capacity dominates the feasibility of running very large models.

GPU and Accelerator Concepts

The lecture assumes familiarity with accelerator execution, but the ideas can be mapped to standard GPU terminology:

- **SM:** Streaming multiprocessor on NVIDIA GPUs. Apple GPUs have a different architecture, but the analogy is useful: many parallel cores execute many independent operations.
- **Global memory:** Large off-chip memory, such as HBM on NVIDIA GPUs or unified memory on Apple Silicon.

- **Memory bandwidth:** Rate at which data can be read from or written to memory.
- **Tensor cores or neural accelerators:** Hardware specialized for dense matrix operations.
- **Interconnect:** Link between devices, such as NVLink, Ethernet, or Thunderbolt.
- **Collective communication:** Operations such as all-reduce, all-gather, reduce-scatter, or point-to-point send and receive.

Distributed Parallelism Vocabulary

The lecture contrasts two major ways of distributing models:

- **Pipeline parallelism:** Split model layers across devices.
- **Tensor parallelism:** Split tensors inside each layer across devices.

For a model with L layers and D devices, pipeline parallelism reduces per-device memory by assigning roughly L/D layers per device. Tensor parallelism reduces per-device tensor size by sharding matrix multiplications, attention projections, and MLP weights across devices.

Main Concepts

Why Run LLMs on Consumer Devices?

The lecture begins by contrasting early GPT-3-era assumptions with the on-device future. During the GPT-3 era, large models were so large that the only realistic option was datacenter inference on NVIDIA GPUs. Users sent prompts and private data to remote servers.

This created a default assumption:

Use LLM $\implies$ send data to someone else's infrastructure.

The lecture argues that this assumption hides many possible use cases. If an assistant could see everything happening on a user's computer, it could become much more useful, but only if privacy concerns were addressed. Local inference enables a different privacy model:

Private data $\xrightarrow{\text{local model}}$ local answer, with no cloud transfer.

Important examples include:

- Personal agents that observe the user's local files, writing, browsing, and workflows.

- Law firms or companies with strict client-data confidentiality requirements.
- On-premise deployments where the data must remain inside a physical office or secure box.
- Hobbyist and research setups where people want to experiment without renting datacenter GPUs.

Why Apple Silicon?

Apple Silicon is interesting because high-end machines provide very large **unified memory**. In the lecture, the device of choice is the M3 Ultra Mac Studio with approximately 512 GB of shared CPU-GPU memory.

The key architectural property is:

CPU memory pool $\approx$ GPU memory pool $\approx$ one unified addressable memory system.

This differs from a traditional discrete GPU setup, where CPU DRAM and GPU VRAM are separate. The unified memory design means that very large models can fit into a single machine more easily, even though compute throughput may be far below an H100 or H200.

The lecture compares a high-end M3 Ultra with an H100-like profile:

- M3 Ultra: much lower peak FLOPs, but very large memory capacity.
- H100 or H200: extremely high peak FLOPs and HBM bandwidth, but much more expensive per gigabyte of memory.

A useful lens is dollar-per-gigabyte:

$$\text{Cost per GB} = \frac{\text{Device cost}}{\text{Usable accelerator memory}}.$$

For large model inference, cost per gigabyte can be more relevant than peak FLOPs because the model must fit before it can run.

The Memory-Capacity Argument

For a model such as DeepSeek with around 670 billion parameters, 8-bit native quantization requires close to 700 GB of memory. This is larger than a single 512 GB Mac Studio, but it can fit across two machines:

$$700 \text{ GB} < 2 \times 512 \text{ GB} = 1024 \text{ GB}.$$

Thus, two high-memory Apple Silicon machines can provide enough memory capacity for a very large model. The lecture contrasts this with an H200 server

box, which may provide much higher throughput but at substantially higher cost and power.

The trade-off is:

Apple cluster = cheap memory and low power + lower tokens per second,

while:

NVIDIA datacenter cluster = high throughput + high cost and high power.

Power as a Practical Constraint

The lecture also discusses power draw. Consumer-device clusters can be placed in an office or home, but power circuits become a real constraint. A cluster of many Mac Studios can draw several kilowatts. In one anecdote, a large rack of Mac Studios tripped office power.

If one device draws approximately P watts and there are D devices, then:

$$P_{\text{cluster}} = DP.$$

For example, if each device draws roughly 500 W under heavy load, then:

$$P_{\text{cluster}} = 16 \times 500 = 8000 \text{ W}.$$

A normal home or office circuit may not support this safely. Thus, consumer distributed ML is not only a software problem; it is also an electrical and physical deployment problem.

Apple Neural Accelerators and Future Direction

The lecture notes that older Apple Silicon chips were not originally designed primarily for large-scale AI. They were designed for workloads such as video editing, photo processing, and local media acceleration. However, newer Apple chips have increasingly emphasized AI acceleration.

The speaker distinguishes between:

- Older separate neural engine blocks.
- Newer architectures that place neural accelerator-like hardware closer to GPU cores.

This resembles the role of NVIDIA tensor cores:

Dense matrix multiply $\implies$ specialized matrix hardware.

The forward-looking argument is that if Apple keeps increasing matrix throughput while preserving large unified memory, future Apple Silicon devices could become much more attractive for local AI workloads.

Apple MLX

MLX as Apple's Machine Learning Framework

Apple MLX is described as Apple's answer to PyTorch for Apple Silicon. It is designed around Apple hardware and has different semantics from PyTorch.

The biggest difference emphasized in the lecture is that MLX uses **lazy execution**. This makes MLX feel closer to JAX in some respects.

Eager Execution in PyTorch

In PyTorch, when a tensor operation is invoked, the operation is usually scheduled immediately. For example:

```
z = x @ w
y = torch.tanh(z)
```

After the first line, PyTorch has scheduled or executed the matrix multiplication. After the second line, it has scheduled or executed the activation. The framework behaves eagerly: operations happen in the order the program requests them, subject to asynchronous device execution.

Conceptually:

$$z = xW, \quad y = \tanh(z).$$

The graph may be retained for autograd, but the operations have already been launched.

Lazy Execution in MLX

In MLX, the equivalent code does not immediately compute z or y . Instead, MLX builds a computational graph:

```
z = x @ w
y = mx.tanh(z)
mx.eval(y)
```

Before `mx.eval(y)`, the tensor y is more like a promise:

$$y \equiv \text{“the result of applying } \tanh \text{ to } xW\text{”}.$$

When `mx.eval(y)` is called, MLX traverses the dependency graph and schedules the necessary operations.

The graph is:

$$x, W \longrightarrow z = xW \longrightarrow y = \tanh(z).$$

Then evaluation realizes the requested output:

$$\text{mx.eval}(y) \implies \text{materialize } y \text{ on device.}$$

Benefits of Lazy Execution

Lazy execution offers several benefits:

- **Avoiding redundant compute:** If intermediate tensors are never evaluated, they may not need to be materialized.
- **Graph-level scheduling:** MLX can see a larger computation graph before launching kernels.
- **Lower peak memory:** The runtime can schedule operations to reduce unnecessary intermediate storage.
- **More natural prefill optimization:** One can evaluate only the needed outputs, such as the key-value cache.

For a chain of operations:

$$a \rightarrow b \rightarrow c \rightarrow d,$$

if only d is evaluated, MLX only needs to materialize what is necessary for d . Intermediate values can sometimes be fused, eliminated, or scheduled efficiently.

LLM Prefill and Decode Under Lazy Execution

During decode, the model computes logits for the next token:

$$\ell_t = f_\theta(x_t, \text{KV}_{<t}),$$

then samples:

$$x_{t+1} \sim \text{Sample}(\text{softmax}(\ell_t)).$$

In code-like form:

```
logits = model(x, cache=kv_cache)
token = sample(logits)
mx.eval(token)
```

In decode, the token must be realized because generation depends on the sampled token.

During prefill, however, one often only needs the key-value cache:

$$\text{KV}_{1:S} = \text{Prefill}_{\theta}(x_{1:S}).$$

The logits at the end of prefill may not be necessary if the system only wants to build the cache. With MLX lazy execution, one can evaluate only the key-value cache:

```
logits, kv_cache = model(prompt_tokens)
mx.eval(kv_cache)
```

The conceptual benefit is:

Evaluate KV cache $\nrightarrow$ necessarily evaluate final output head.

In PyTorch, this often requires modifying the model forward function with a flag such as `return_kv_only=True`. In MLX, the requested evaluated object controls what is computed.

MLX Compilation

MLX also supports compilation through a function transformation, similar in spirit to JAX transformations. The lecture contrasts naive execution with compiled execution.

A simplified example:

```
def f(x, w):
    z = x @ w
    z = mx.tanh(z)
    z = z + 1.0
    return z
```

```
compiled_f = mx.compile(f)
```

```
y = compiled_f(x, w)
mx.eval(y)
```

Without compilation, the graph might produce several separate kernel launches:

matmul $\rightarrow$ tanh $\rightarrow$ add $\rightarrow$ store.

With compilation, compatible elementwise operations can be fused:

matmul $\rightarrow$ fused tanh-add-store.

Kernel fusion reduces:

- kernel launch overhead,
- intermediate memory traffic,
- global memory writes and reads,
- synchronization between kernels.

The performance improvement comes from reducing memory movement. If an intermediate tensor of n elements is written to memory and later read back, then for dtype size s bytes the extra traffic is:

$$\text{Extra bytes} = 2ns.$$

Fusion can avoid this intermediate round trip.

Shape Sensitivity

The lecture warns that compilation must handle changing shapes in LLM inference. During generation, sequence length and batch shapes can vary. A compiled graph may specialize to particular shapes. If the shapes change, the framework may need recompilation or special dynamic-shape handling.

This is especially important for autoregressive models:

$$S_{\text{decode}} = 1, \quad S_{\text{prefill}} = \text{prompt length},$$

so prefill and decode often have very different shapes and performance profiles.

Inference Clusters

Single-Device MLX-LM

MLX-LM is Apple's language model inference library for MLX. It allows users to run Hugging Face-style models on Apple Silicon.

The single-device limitation is memory:

Model bytes + KV cache bytes + runtime overhead $\leq$ device memory.

When this inequality fails, the model cannot fit on one device. Consumer-device clustering attempts to solve this by aggregating memory across machines:

$$\text{Total memory} = \sum_{i=1}^D M_i.$$

For D identical devices each with memory M :

$$\text{Total memory} = DM.$$

Exo-Style Inference Clusters

The lecture describes ExoLabs as building orchestration software for consumer-device clusters. The product idea is:

many Macs+auto-discovery+distributed inference runtime $\implies$ larger local model serving.

The cluster software needs to handle:

- discovery of devices,
- model partitioning,
- communication between ranks,
- fault tolerance,
- orchestration of inference requests,
- local device failures or unplugged cables.

MLX distributed provides low-level send and receive operations, but an orchestration layer is needed to make a robust product.

Pipeline Parallelism

Layer-Wise Partitioning

Pipeline parallelism splits model layers across devices. Suppose a model has L layers and D devices. Device i receives a contiguous subset of layers:

$$\mathcal{L}_i = \left\{ \ell : \left\lfloor \frac{iL}{D} \right\rfloor \leq \ell < \left\lfloor \frac{(i+1)L}{D} \right\rfloor \right\}.$$

For a toy model with four layers and two devices:

$$\text{Device 0 : } \{1, 2\}, \quad \text{Device 1 : } \{3, 4\}.$$

The forward pass is:

$$h_0 = x,$$

$$h_2 = f_2(f_1(h_0)),$$

$$h_4 = f_4(f_3(h_2)).$$

The communication boundary is the activation tensor h_2 , sent from device 0 to device 1.

Communication Cost

For two devices and one forward pass, the main communication is:

$$\text{Device 0} \rightarrow \text{Device 1} : h_2.$$

For autoregressive generation, the final token or output may need to return to the first device or coordinator. Thus the lecture describes roughly two communications per token in a simple two-device pipeline:

$$N_{\text{comm,pipeline}} \approx O(D).$$

For $D = 2$:

$$N_{\text{comm,pipeline}} \approx 2.$$

The major drawback is poor device utilization for a single request. While device 0 is computing, device 1 may wait. Then device 1 computes while device 0 waits.

For a single token and no batching, utilization resembles:

$$U_i \approx \frac{T_i}{\sum_{j=1}^D T_j + T_{\text{comm}}},$$

where T_i is local compute time on device i .

Pipeline Bubble

Pipeline parallelism has idle time known as the **pipeline bubble**. With micro-batching during training, the bubble can be reduced, but during single-request autoregressive inference, the computation is often sequential.

If each device stage takes time T and there are D stages, one token may take approximately:

$$T_{\text{token}} \approx DT + T_{\text{comm}}.$$

This means adding devices increases memory capacity, but does not necessarily improve tokens per second. It can even reduce throughput if communication is high.

Visual Concept

Draw a vertical stack of transformer layers. Put layers 1 and 2 inside a box labeled **Mac 0**, and layers 3 and 4 inside a box labeled **Mac 1**. Draw an activation arrow from **Mac 0** to **Mac 1**, then a return arrow for the output token.

Tensor Parallelism

Sharding Inside Layers

Tensor parallelism cuts through the model vertically. Instead of assigning whole layers to different devices, it shards the tensors inside each layer.

For a linear layer:

$$Y = XW,$$

where:

$$X \in \mathbb{R}^{B \times d_{\text{in}}}, \quad W \in \mathbb{R}^{d_{\text{in}} \times d_{\text{out}}}, \quad Y \in \mathbb{R}^{B \times d_{\text{out}}}.$$

A column-parallel split shards W across output channels:

$$W = [W_1 \quad W_2 \quad \cdots \quad W_D],$$

so each device computes:

$$Y_i = XW_i.$$

Then:

$$Y = \begin{bmatrix} Y_1 & Y_2 & \cdots & Y_D \end{bmatrix}.$$

A row-parallel split shards input channels:

$$W = \begin{bmatrix} W_1 \\ W_2 \\ \vdots \\ W_D \end{bmatrix}, \quad X = \begin{bmatrix} X_1 & X_2 & \cdots & X_D \end{bmatrix},$$

so each device computes a partial result:

$$Z_i = X_i W_i.$$

Then an all-reduce or reduce-sum is needed:

$$Y = \sum_{i=1}^D Z_i.$$

Communication Frequency

The lecture emphasizes that tensor parallelism requires more frequent communication than pipeline parallelism. In a transformer block, an efficient tensor-parallel layout can often reduce communication to about two communications per layer.

For a model with L layers:

$$N_{\text{comm}, \text{tensor}} \approx 2L.$$

For DeepSeek with $L = 61$ layers:

$$N_{\text{comm}, \text{tensor}} \approx 2 \times 61 = 122.$$

This is acceptable only if each communication is low latency. Otherwise, communication dominates.

Latency Model

Let:

$$\tau = \text{latency per communication hop.}$$

Then tensor-parallel latency overhead per generated token is approximately:

$$T_{\text{comm,tensor}} \approx 2L\tau.$$

For $\tau = 0.5$ ms and $L = 61$:

$$T_{\text{comm,tensor}} \approx 2 \times 61 \times 0.5 \text{ ms} = 61 \text{ ms}.$$

If pipeline inference already generates at about 15 tokens per second, then:

$$T_{\text{token}} \approx \frac{1}{15} \approx 66.7 \text{ ms}.$$

Thus a 61 ms communication overhead is catastrophic:

$$T_{\text{comm,tensor}} \approx T_{\text{token}}.$$

The model would spend roughly as much time communicating as computing.

Impact of RDMA Over Thunderbolt

The lecture notes that newer macOS support for RDMA over Thunderbolt can reduce latency dramatically. If the latency falls to around 5 microseconds:

$$\tau = 5\mu s = 0.005 \text{ ms}.$$

Then:

$$T_{\text{comm,tensor}} \approx 2 \times 61 \times 0.005 \text{ ms} = 0.61 \text{ ms}.$$

This changes tensor parallelism from infeasible to plausible:

$$0.61 \text{ ms} \ll 66.7 \text{ ms}.$$

The same tensor-parallel algorithm can be impossible or practical depending almost entirely on the communication latency of the interconnect.

Tensor Parallelism Versus Pipeline Parallelism

The design trade-off is:

Pipeline parallelism = low communication frequency+poor simultaneous compute utilization.

Tensor parallelism = high communication frequency+better simultaneous compute utilization.

Pipeline parallelism primarily increases memory capacity. Tensor parallelism can increase throughput if communication latency is low enough.

Visual Concept

Draw the transformer as a stack of layers. For pipeline parallelism, cut horizontally between layers. For tensor parallelism, cut vertically through every layer, showing each layer split across multiple devices. Label the vertical split as requiring synchronization after each layer.

Physical Topology and Thunderbolt

Thunderbolt as a Point-to-Point Interconnect

The lecture highlights that Apple Silicon clusters are physically different from NVIDIA servers. NVIDIA servers often have carefully engineered NVLink topologies. Apple devices connect through Thunderbolt or Ethernet.

Thunderbolt 5 is described as a high-bandwidth point-to-point link. The important part is **point-to-point**. If a communication pattern requires device i to talk directly to device j , it is best if there is a physical cable connecting them.

This means the physical wiring becomes part of the distributed algorithm.

Algorithmic topology $\approx$ Cable topology.

For a ring topology:

$$0 \rightarrow 1 \rightarrow 2 \rightarrow \dots \rightarrow D-1 \rightarrow 0.$$

Each device only needs to communicate with its predecessor and successor. This maps naturally to a physical ring of Thunderbolt cables.

Multiple Parallelism Dimensions

If a Mac has six Thunderbolt ports, one can imagine multiple dimensions of connectivity. A single ring requires two ports per device:

$$\text{ports used per ring} = 2.$$

With six ports, in principle, one could support three independent ring-like communication dimensions:

$$\frac{6}{2} = 3.$$

This motivates speculative combinations such as:

pipeline parallelism + tensor parallelism + context parallelism.

Each dimension may require a different communication pattern.

Why Topology Matters More on Consumer Hardware

On datacenter systems, the programmer often relies on high-level collectives such as NCCL all-reduce. The physical topology is still important, but it is hidden behind optimized libraries and switches.

On consumer-device clusters, the developer may directly experience:

- cable placement,
- link bandwidth differences,
- link failures,
- blocking send/receive deadlocks,
- rank mismatch bugs,
- routing constraints.

This makes the system feel more bare-metal and educational.

Visual Concept

Draw four Mac Studios arranged in a square. Connect them with Thunderbolt cables in a ring. Then draw a second diagram where every device wants all-to-all communication and show that the cable topology becomes much more complex.

Heterogeneous Prefill and Decode

Different Hardware Profiles

The lecture discusses a heterogeneous setup involving an M3 Ultra and an NVIDIA DGX Spark-like device. The key observation is that different devices can have different roofline profiles:

M3 Ultra = high memory capacity and strong bandwidth relative to FLOPs,

DGX Spark = higher compute throughput but lower memory bandwidth profile.

This creates a crossover point in the roofline model. Let device A be the M3 Ultra and device B be the DGX Spark. Their roofline bounds are:

$$P_A(I) = \min(P_{A,\text{peak}}, B_A I),$$

$$P_B(I) = \min(P_{B,\text{peak}}, B_B I).$$

The crossover intensity I^* solves:

$$P_A(I^*) = P_B(I^*).$$

The lecture mentions a crossover around:

$$I^* \approx 95 \text{ FLOPs/byte.}$$

If:

$$I < I^*,$$

the M3 Ultra may be better because the workload is memory-bound. If:

$$I > I^*,$$

the DGX Spark may be better because the workload is compute-bound.

Prefill Is Compute-Heavy

Prefill processes many tokens at once. Large matrix multiplications have higher arithmetic intensity because weights are reused across the sequence dimension.

For a matrix multiplication:

$$C = AB,$$

with:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the FLOPs are:

$$\text{FLOPs} = 2MKN.$$

The bytes moved, ignoring cache reuse details, are approximately:

$$\text{Bytes} \approx s(MK + KN + MN),$$

where s is bytes per element. The arithmetic intensity is:

$$I \approx \frac{2MKN}{s(MK + KN + MN)}.$$

As M grows with prompt sequence length, reuse improves and intensity increases. Thus prefill can be compute-bound.

Decode Is Memory-Heavy

Decode has small batch and sequence dimensions. Each token still needs to read large weights, but the amount of computation per loaded byte is lower.

A rough decode arithmetic intensity is:

$$I_{\text{decode}} \approx \frac{\text{FLOPs per token}}{\text{weight bytes read per token}}.$$

Because the weights are huge and the token count is one, I_{decode} is often low. Thus decode is memory-bound.

Disaggregated Prefill and Decode

The heterogeneous strategy is:

Prefill on compute-heavy device $\rightarrow$ stream KV cache $\rightarrow$ Decode on memory-heavy device.

For example:

$$\text{DGX Spark} \xrightarrow{\text{KV cache over network}} \text{M3 Ultra}.$$

This can reduce end-to-end latency if:

$$T_{\text{prefill,Spark}} + T_{\text{KV transfer}} + T_{\text{decode,Mac}} < T_{\text{prefill,Mac}} + T_{\text{decode,Mac}}.$$

The difficult term is $T_{\text{KV transfer}}$. If the devices communicate over 10 Gb Ethernet, the bandwidth is much lower than local memory bandwidth or Thunderbolt.

If the KV cache has size K_{bytes} and network bandwidth is B_{net} , then:

$$T_{\text{KV transfer}} = \frac{K_{\text{bytes}}}{B_{\text{net}}}.$$

To make this practical, transfer must be overlapped with prefill computation:

$$T_{\text{effective}} \approx \max(T_{\text{prefill}}, T_{\text{KV transfer}}) + T_{\text{decode}}.$$

Without overlap, transfer can erase the benefit.

Visual Concept

Draw two roofline curves on the same axes. Label the x-axis arithmetic intensity and the y-axis FLOPs per second. Show a crossover near 95 FLOPs/byte. Mark decode on the left side and prefill on the right side.

Training on Consumer-Device Clusters

Why Training Seems Unnatural on Macs

The lecture calls training on Mac clusters somewhat crazy because training is much more compute-intensive than inference. Training requires:

- forward pass,
- backward pass,

- gradient storage,
- optimizer state,
- parameter updates,
- often activation checkpointing.

For full-parameter training with Adam, memory can be several times parameter memory. If the model parameters use P bytes, gradients use another P , and Adam moment estimates use roughly $2P$, then optimizer-related storage can be:

$$\text{Training memory} \approx P_{\text{params}} + P_{\text{grads}} + 2P_{\text{Adam states}} + P_{\text{activations}}.$$

More explicitly, if parameters, gradients, and two optimizer states all have similar size:

$$M_{\text{train}} \approx 4P + M_{\text{activations}}.$$

For a 700 GB model, full training is infeasible on two 512 GB devices. This motivates LoRA.

Why Train Anyway?

The lecture gives several reasons to attempt training:

- Inference clusters may sit idle overnight.
- Future Apple chips may have much higher AI throughput.
- The space is underexplored.
- It is valuable to learn how far the hardware and framework can be pushed.

The philosophy is:

Use idle local inference hardware $\implies$ perform small fine-tuning jobs.

LoRA as the Practical Fine-Tuning Method

LoRA reduces trainable parameter count by freezing the base model and training low-rank adapters. For a weight matrix:

$$W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}},$$

LoRA represents an update as:

$$\Delta W = BA,$$

where:

$$B \in \mathbb{R}^{d_{\text{out}} \times r}, \quad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}).$$

The adapted weight is:

$$W' = W + \alpha BA.$$

The trainable parameter count is:

$$P_{\text{LoRA}} = r(d_{\text{in}} + d_{\text{out}}),$$

instead of:

$$P_{\text{full}} = d_{\text{in}} d_{\text{out}}.$$

The ratio is:

$$\frac{P_{\text{LoRA}}}{P_{\text{full}}} = \frac{r(d_{\text{in}} + d_{\text{out}})}{d_{\text{in}} d_{\text{out}}}.$$

If $d_{\text{in}} = d_{\text{out}} = d$, then:

$$\frac{P_{\text{LoRA}}}{P_{\text{full}}} = \frac{2r}{d}.$$

For $d = 8192$ and $r = 16$:

$$\frac{P_{\text{LoRA}}}{P_{\text{full}}} = \frac{32}{8192} \approx 0.0039.$$

Thus LoRA can train less than 1% of the full model parameters.

Backpropagation in MLX

PyTorch Autograd Mental Model

In PyTorch, tensors can carry autograd metadata. During the forward pass, PyTorch builds a computation graph. Calling:

```
loss.backward()
```

traverses the graph backward and accumulates gradients into parameter fields such as:

```
param.grad
```

A simplified training loop is:

```
optimizer.zero_grad()
logits = model(tokens)
loss = cross_entropy(logits, targets)
loss.backward()
optimizer.step()
```

Conceptually:

$$\theta \rightarrow \ell(\theta) \rightarrow \nabla_{\theta} \ell.$$

MLX Value-and-Grad

MLX uses a functional transformation called `value_and_grad`. Instead of attaching gradients as mutable fields to tensors, one transforms a scalar loss function into a function that returns both the loss and gradients.

Let:

$$f(\theta, x) = \ell,$$

where ℓ is a scalar loss. Then MLX constructs:

$$g(\theta, x) = (f(\theta, x), \nabla_{\theta} f(\theta, x)).$$

In code-like form:

```
def loss_fn(model, batch):
    logits = model(batch["tokens"])
    return cross_entropy(logits, batch["targets"])

loss_and_grad_fn = mx.value_and_grad(loss_fn)
```

```
loss, grads = loss_and_grad_fn(model, batch)
mx.eval(loss, grads)
```

The important point is that MLX builds one graph containing the forward and backward computation, then evaluates it lazily.

Forward-Backward Graph

For a simple MLP:

$$z = xW, \quad a = \tanh(z), \quad \ell = \mathcal{L}(a, y).$$

Backprop computes:

$$\frac{\partial \ell}{\partial a}, \quad \frac{\partial \ell}{\partial z}, \quad \frac{\partial \ell}{\partial W}.$$

Using the chain rule:

$$\frac{\partial \ell}{\partial z} = \frac{\partial \ell}{\partial a} \odot (1 - \tanh^2(z)).$$

For the matrix multiplication:

$$z = xW,$$

the weight gradient is:

$$\frac{\partial \ell}{\partial W} = x^\top \frac{\partial \ell}{\partial z}.$$

The input gradient is:

$$\frac{\partial \ell}{\partial x} = \frac{\partial \ell}{\partial z} W^\top.$$

MLX can construct this full graph through the transformation.

Pipeline-Parallel Training

Global View

For pipeline-parallel training, each rank owns a subset of layers. Suppose there are three devices:

Device 1 $\rightarrow$ Device 2 $\rightarrow$ Device 3.

The forward pass sends intermediate activations:

$$h_1 \rightarrow h_2 \rightarrow h_3.$$

The final device computes the loss:

$$\ell = \mathcal{L}(h_3, y).$$

The backward pass moves in reverse:

$$\frac{\partial \ell}{\partial h_3} \rightarrow \frac{\partial \ell}{\partial h_2} \rightarrow \frac{\partial \ell}{\partial h_1}.$$

Each device computes local parameter gradients. If device i owns parameters θ_i , then it computes:

$$\nabla_{\theta_i} \ell.$$

Boundary Gradients

Consider a middle device i . It receives an activation h_{i-1} from the previous device, computes:

$$h_i = F_i(h_{i-1}; \theta_i),$$

then sends h_i forward.

During backprop, it receives:

$$\bar{h}_i = \frac{\partial \ell}{\partial h_i}.$$

It must compute:

$$\nabla_{\theta_i} \ell = \frac{\partial \ell}{\partial \theta_i},$$

and also the gradient to send backward:

$$\bar{h}_{i-1} = \frac{\partial \ell}{\partial h_{i-1}}.$$

Using the chain rule:

$$\bar{h}_{i-1} = \bar{h}_i \frac{\partial h_i}{\partial h_{i-1}}.$$

The MLX Problem: No Direct Backward From Arbitrary Gradient

In PyTorch, one can often backpropagate from an intermediate tensor with an externally supplied gradient:

```
y.backward(gradient=dy)
```

This means:

$$\frac{\partial \ell}{\partial y} = dy.$$

In MLX, the primary interface is `value_and_grad`, which expects a scalar loss. Therefore, if a middle pipeline rank receives an external gradient $\bar{h}_i$, it needs to construct a scalar surrogate loss whose gradient with respect to h_i equals $\bar{h}_i$.

Auxiliary Loss Trick

The goal is to create $\hat{\ell}$ such that:

$$\frac{\partial \hat{\ell}}{\partial h_i} = \bar{h}_i.$$

A simple choice is the dot product:

$$\hat{\ell} = \langle h_i, \bar{h}_i \rangle.$$

Expanded elementwise:

$$\hat{\ell} = \sum_j h_{i,j} \bar{h}_{i,j}.$$

Then:

$$\frac{\partial \hat{\ell}}{\partial h_{i,j}} = \bar{h}_{i,j}.$$

Therefore:

$$\nabla_{h_i} \hat{\ell} = \bar{h}_i.$$

This scalar fake loss allows MLX to use its normal scalar-loss differentiation path while implementing pipeline backprop from an arbitrary received gradient.

The auxiliary loss $\hat{\ell} = \langle h_i, \bar{h}_i \rangle$ is not a real training objective. It is a mathematical shim that makes MLX propagate exactly the boundary gradient received from the next pipeline stage.

Representative MLX-Style Pseudocode

```
def local_forward(params, incoming_activation):
    hidden = stage_layers(params, incoming_activation)
    return hidden

def fake_loss_fn(params, incoming_activation, incoming_grad):
    outgoing_activation = local_forward(params, incoming_activation)
    fake_loss = mx.sum(outgoing_activation * incoming_grad)
    return fake_loss

loss_and_grad_fn = mx.value_and_grad(fake_loss_fn)

fake_loss, local_grads = loss_and_grad_fn(
    params,
    incoming_activation,
    incoming_grad,
)

mx.eval(fake_loss, local_grads)
```

The major line-by-line meaning is:

- `local_forward` computes this rank’s subset of transformer layers.
- `incoming_activation` is the residual stream received from the previous rank.
- `incoming_grad` is $\partial \ell / \partial h_i$ received from the next rank.
- `mx.sum(outgoing_activation * incoming_grad)` builds the auxiliary scalar loss.
- `mx.value_and_grad` differentiates this scalar with respect to local parameters.
- `mx.eval` realizes the lazy graph.

Hardware behavior:

- The local forward pass runs on the local Apple GPU.

- The auxiliary loss is an elementwise multiply plus reduction.
- Backward kernels compute LoRA or local parameter gradients.
- Inter-device communication supplies the boundary activation and boundary gradient.

Deadlocks and Artificial Dependencies

The Lazy Distributed Graph Problem

The lecture gives a subtle MLX distributed training issue. In a pipeline-parallel rank, the graph may contain two receive operations:

$$\text{recv}(h_{i-1})$$

for the forward pass, and:

$$\text{recv}(\bar{h}_i)$$

for the backward pass.

It also contains two send operations:

$$\text{send}(h_i), \quad \text{send}(\bar{h}_{i-1}).$$

From the local graph's perspective, these may look like independent leaves. MLX may not know that the backward receive must occur only after the forward send.

If a rank tries to receive $\bar{h}_i$ before sending h_i , it can block forever because the next rank cannot produce $\bar{h}_i$ until it receives h_i and computes forward and backward.

This creates a deadlock.

Deadlock Condition

Let rank i perform:

$$R_i^f = \text{recv}(h_{i-1}), \quad S_i^f = \text{send}(h_i),$$

$$R_i^b = \text{recv}(\bar{h}_i), \quad S_i^b = \text{send}(\bar{h}_{i-1}).$$

The required order is:

$$R_i^f \rightarrow \text{forward compute} \rightarrow S_i^f \rightarrow R_i^b \rightarrow \text{backward compute} \rightarrow S_i^b.$$

If MLX schedules:

$$R_i^b \text{ before } S_i^f,$$

then rank i waits for a gradient that cannot exist yet.

Artificial Dependency Solution

The solution is to create an artificial dependency:

$$R_i^b \text{ depends on } S_i^f.$$

In words:

Do not receive the backward gradient until the forward activation has been sent.

This forces the graph ordering:

$$S_i^f \rightarrow R_i^b.$$

The broader lesson is that in a lazy distributed runtime, local graph correctness is not enough. The ranks must collectively implement a consistent global protocol.

Global Communication Protocol

Every distributed program has an implicit communication protocol. For point-to-point communication, the protocol must specify:

- which rank sends,
- which rank receives,
- what tensor shape is sent,
- what dtype is sent,
- when the send occurs,
- when the receive occurs,
- whether the operation is blocking,
- what dependency ordering must hold.

If rank i sends a tensor of shape (B, d) but rank j expects $(B, 2d)$, the framework may not detect the error until runtime. If both ranks block on receives, the program deadlocks.

Local type correctness $\nrightarrow$ global distributed correctness.

Hardware-Level Explanation

Apple Unified Memory

Apple Silicon uses a unified memory architecture. CPU and GPU share a memory pool, which avoids explicit CPU-to-GPU copies in many cases. For large models, the main advantage is capacity.

On a discrete NVIDIA GPU system:

CPU DRAM $\neq$ GPU HBM.

On Apple Silicon:

CPU-visible memory $\approx$ GPU-visible memory.

This enables enormous local model residency compared with many consumer discrete GPUs.

Why FLOPs Alone Mislead

A device with 900 TFLOPs and 80 GB of memory may be unusable for a 700 GB model without sharding. A device with 26 TFLOPs and 512 GB of memory may be slower but can hold much more of the model.

The feasibility condition is:

$$M_{\text{model}} + M_{\text{KV}} + M_{\text{runtime}} \leq M_{\text{device}}.$$

Only after this condition is satisfied does peak FLOP throughput matter.

Decode on Memory-Balanced Hardware

During decode, each token reads model weights and KV cache. If the model is quantized to b bytes per parameter, the lower-bound memory traffic per token is:

$$T_{\text{bytes}} \geq N_{\text{params}} b.$$

The maximum possible token rate from memory bandwidth alone is approximately:

$$\text{TPS}_{\max} \leq \frac{B_{\text{mem}}}{N_{\text{params}}b}.$$

This is a simplified bound because real inference also includes KV cache traffic, activations, dequantization, synchronization, and kernel overhead. But it explains why memory bandwidth and model size dominate decode.

Interconnect Latency Versus Bandwidth

Distributed LLM inference has two communication dimensions:

- **Latency:** Cost to begin or complete a communication event.
- **Bandwidth:** Rate for moving large tensors.

Tensor parallelism is often latency-sensitive because it communicates many times per token:

$$N_{\text{comm}} \sim 2L.$$

Pipeline parallelism is less latency-sensitive but can be bandwidth-sensitive if activation tensors are large.

The time to send a message of size S bytes is:

$$T_{\text{message}} = \tau + \frac{S}{B},$$

where:

- τ is latency,
- B is interconnect bandwidth.

For small messages, latency dominates. For large activation or KV-cache transfers, bandwidth dominates.

NVIDIA Versus Consumer Interconnects

NVIDIA systems often use NVLink, which is optimized for high bandwidth and low latency within a server. Consumer clusters may use:

- Ethernet,
- Thunderbolt,
- USB-like physical connectors with Thunderbolt protocol,

- custom device-to-device links for specific boxes.

Thus, distributed strategies designed for NVLink do not always transfer directly.

On NVIDIA, tensor parallelism inside a box is natural because NVLink makes frequent communication cheap. On consumer-device clusters, the same communication pattern can be prohibitive unless RDMA or a very low-latency link is available.

Mathematical Reasoning

Model Memory

For N parameters and q bits per parameter:

$$M_{\text{weights}} = \frac{Nq}{8} \text{ bytes.}$$

For $N = 670 \times 10^9$ and $q = 8$:

$$M_{\text{weights}} = \frac{670 \times 10^9 \times 8}{8} = 670 \times 10^9 \text{ bytes} \approx 670 \text{ GB.}$$

For 4-bit quantization:

$$M_{\text{weights}} = \frac{670 \times 10^9 \times 4}{8} = 335 \times 10^9 \text{ bytes} \approx 335 \text{ GB.}$$

This explains why 4-bit quantized DeepSeek can fit on a single 512 GB machine, while 8-bit may require two.

KV Cache Memory

For a transformer with:

- L layers,
- batch size B ,
- sequence length S ,
- number of KV heads H_{kv} ,
- head dimension d_h ,
- dtype size s bytes,

the KV cache stores keys and values:

$$M_{KV} = 2LBSh_{kv}d_h s.$$

The factor 2 accounts for keys and values.

This grows linearly with sequence length:

$$M_{KV} \propto S.$$

Therefore, long-context inference can become memory-limited even if the model weights fit.

Pipeline Communication

If each pipeline boundary activation has size A bytes and there are $D - 1$ boundaries, then one forward pass sends:

$$M_{\text{comm,forward}} = (D - 1)A.$$

For backward training, gradients of similar size are sent backward:

$$M_{\text{comm,backward}} = (D - 1)A.$$

Thus, for training:

$$M_{\text{comm,total}} \approx 2(D - 1)A.$$

For inference decode with one token, A may be relatively small, but latency dominates.

Tensor Parallel Communication

For a transformer with L layers and two communications per layer:

$$N_{\text{messages}} = 2L.$$

The latency-only communication time is:

$$T_{\text{latency}} = 2L\tau.$$

The bandwidth cost depends on the size of the hidden activation or partial result. If each communication transfers A bytes:

$$T_{\text{bandwidth}} = 2L \frac{A}{B}.$$

Total communication time:

$$T_{\text{comm}} = 2L \left(\tau + \frac{A}{B} \right).$$

When Tensor Parallelism Helps

Let one device compute time per token be T_1 . Suppose tensor parallelism across D devices ideally reduces compute time to:

$$T_{\text{compute},D} = \frac{T_1}{D}.$$

Then tensor parallelism helps if:

$$\frac{T_1}{D} + T_{\text{comm},D} < T_1.$$

Equivalently:

$$T_{\text{comm},D} < T_1 \left(1 - \frac{1}{D} \right).$$

For $D = 2$:

$$T_{\text{comm},2} < \frac{T_1}{2}.$$

If communication exceeds half the single-device compute time, two-way tensor parallelism will not speed up inference.

Auxiliary Loss Derivation

Given a desired incoming gradient:

$$\bar{y} = \frac{\partial \ell}{\partial y},$$

construct:

$$\hat{\ell}(y) = \sum_j y_j \bar{y}_j.$$

Then:

$$\frac{\partial \hat{\ell}}{\partial y_k} = \frac{\partial}{\partial y_k} \sum_j y_j \bar{y}_j = \bar{y}_k.$$

Therefore:

$$\nabla_y \hat{\ell} = \bar{y}.$$

This is why the fake scalar loss works.

Code Walkthrough

Lazy Evaluation Example

```
import mlx.core as mx

x = mx.random.normal((1024, 1024))
w = mx.random.normal((1024, 1024))

z = x @ w
y = mx.tanh(z)

mx.eval(y)
```

Explanation:

- `x` and `w` are MLX arrays.
- `z = x @ w` creates a graph node for matrix multiplication.
- `y = mx.tanh(z)` creates a graph node for the activation.
- `mx.eval(y)` forces realization of `y`.

Hardware implications:

- The matrix multiplication is the dominant compute kernel.
- The tanh operation is elementwise and memory-sensitive.
- If compiled or fused, MLX may reduce intermediate memory traffic.

Prefill-Only Evaluation

```
logits, kv_cache = model(prompt_tokens)
mx.eval(kv_cache)
```

Explanation:

- The model call builds a graph for logits and KV cache.
- The program asks only for the KV cache to be realized.
- MLX can avoid unnecessary work associated with outputs that are not needed.

This is useful because prefill often needs to populate:

$$K_{1:S}, V_{1:S},$$

but not necessarily produce final logits.

Compilation and Fusion

```
def block(x, w, b):
    y = x @ w
    y = y + b
    y = mx.maximum(y, 0.0)
    return y
```

```
compiled_block = mx.compile(block)
```

```
out = compiled_block(x, w, b)
mx.eval(out)
```

The unfused operations are:

$$xW \rightarrow +b \rightarrow \text{ReLU} \rightarrow \text{store}.$$

The bias and ReLU are good fusion candidates:

$$\text{ReLU}(xW + b).$$

Why this matters:

- Without fusion, $xW + b$ may be written to memory and read back.
- With fusion, the elementwise operations can be applied before final storage.
- This reduces global memory traffic and kernel launches.

Pipeline Send and Receive Pseudocode

```
def pipeline_stage(params, rank, world_size):
    if rank == 0:
        h = load_input_tokens()
    else:
```

```

    h = recv(rank - 1)

h_out = local_layers(params, h)

if rank != world_size - 1:
    send(h_out, rank + 1)
else:
    logits = output_head(h_out)
    token = sample(logits)
    return token

```

Explanation:

- Rank 0 receives user input.
- Intermediate ranks receive activations from the previous rank.
- Each rank runs its local layers.
- Non-final ranks send activations to the next rank.
- The final rank computes logits and samples or returns a token.

Communication behavior:

- Each stage communicates only with adjacent stages.
- This maps naturally to a line or ring topology.
- Device utilization is poor for a single request unless multiple requests or microbatches are pipelined.

Tensor-Parallel Linear Layer Pseudocode

```

def column_parallel_linear(x, w_shard):
    y_shard = x @ w_shard
    return y_shard

def row_parallel_linear(x_shard, w_shard, group):
    partial = x_shard @ w_shard
    y = all_reduce_sum(partial, group=group)
    return y

```

Explanation:

- Column-parallel linear layers split output features.
- Row-parallel linear layers split input features.
- Row-parallel layers require a sum across devices.
- The all-reduce is the expensive synchronization point.

Hardware behavior:

- Each device computes a smaller matrix multiplication.
- The communication pattern determines scalability.
- On low-latency interconnects, the parallel compute can outweigh communication.
- On high-latency interconnects, the all-reduce dominates.

Auxiliary Loss for Pipeline Backward

```
def fake_boundary_loss(local_params, h_in, grad_h_out):
    h_out = local_stage(local_params, h_in)
    return mx.sum(h_out * grad_h_out)

loss_and_grad = mx.value_and_grad(fake_boundary_loss)

fake_loss, grads = loss_and_grad(
    local_params,
    h_in,
    grad_h_out,
)

mx.eval(fake_loss, grads)
```

This implements:

$$\hat{\ell} = \langle h_{\text{out}}, \bar{h}_{\text{out}} \rangle.$$

The gradient with respect to h_{out} is exactly:

$$\nabla_{h_{\text{out}}} \hat{\ell} = \bar{h}_{\text{out}}.$$

This lets a middle pipeline stage start backpropagation from a received boundary gradient.

Artificial Dependency Pseudocode

```
h_in = recv(prev_rank)
h_out = local_forward(params, h_in)

send_token = send(h_out, next_rank)

grad_h_out = recv(next_rank, depends_on=send_token)

fake_loss = mx.sum(h_out * grad_h_out)
loss, grads = value_and_grad(fake_loss)
```

The key idea is:

$$\text{recv}(\bar{h}_{\text{out}}) \text{ must depend on } \text{send}(h_{\text{out}}).$$

Without this dependency, a lazy scheduler may choose the backward receive too early.

Performance Intuition

Why Apple Silicon Works for Large Local Inference

Apple Silicon works well when the main bottleneck is model residency:

$$\text{Need many GB of memory} > \text{Need maximum TFLOPs.}$$

Large unified memory allows very large quantized models to fit. The relatively favorable memory-bandwidth-to-FLOP ratio helps memory-bound decode.

Why Apple Silicon Is Weak for Heavy Training

Training is more compute-intensive and has more memory overhead. Even if weights fit, training needs gradients and optimizer state:

$$M_{\text{train}} \gg M_{\text{inference}}.$$

Therefore, full training is usually impractical. LoRA makes training possible by reducing trainable parameters.

Why Pipeline Parallelism Is the First Natural Strategy

Pipeline parallelism is simple and robust:

$$\text{split layers} \implies \text{send activations between stages.}$$

It requires few communications per token. This is important when latency is high.

However, pipeline parallelism is not ideal for throughput because only one stage is active at a time for a single decode stream.

Why Tensor Parallelism Becomes Attractive With RDMA

Tensor parallelism lets devices compute the same layer simultaneously. This can improve tokens per second:

$$T_{\text{compute}} \rightarrow \frac{T_{\text{compute}}}{D}.$$

But it requires many communications:

$$N_{\text{comm}} \sim 2L.$$

When latency is 0.5 ms, this is bad. When latency is 5 microseconds, it becomes plausible.

Why Heterogeneous Prefill and Decode Can Help

Prefill and decode have different arithmetic intensities:

$$I_{\text{prefill}} \gg I_{\text{decode}}.$$

Thus, one should not assume the same device is best for both. A compute-heavy device may be better for prefill, while a memory-balanced high-capacity device may be better for decode.

The optimal assignment is:

$$\text{device}^* = \arg \max_d P_d(I_{\text{workload}}).$$

The Real Bottleneck Is Often Software Maturity

The lecture repeatedly implies that consumer-device ML is young. The hardware may be capable, but the software stack is less mature than CUDA plus NCCL plus PyTorch.

Major pain points include:

- distributed deadlocks,
- weak global protocol checking,
- shape mismatch debugging,
- physical topology management,
- fault tolerance,
- compilation with dynamic shapes,
- integrating heterogeneous devices.

Common Misconceptions

Misconception: More FLOPs Always Means Better Inference

For decode, memory traffic can dominate. A lower-FLOP device with better memory capacity or better memory-bandwidth balance can be more useful for very large models.

Correct mental model:

Inference feasibility first depends on memory capacity.

Then performance depends on bandwidth, compute, quantization, kernels, and communication.

Misconception: Pipeline Parallelism Speeds Up Single-Request Decode

Pipeline parallelism mostly increases model capacity. For a single request, it often introduces idle stages.

Correct mental model:

Pipeline parallelism $\approx$ memory scaling unless microbatching or concurrency hides bubbles.

Misconception: Tensor Parallelism Is Always Better

Tensor parallelism improves compute utilization but communicates frequently. It only helps when:

$$T_{\text{communication}} < \text{compute time saved.}$$

On high-latency links, tensor parallelism can be slower than pipeline parallelism.

Misconception: MLX Works Just Like PyTorch

MLX is lazy and functional. Tensors do not behave exactly like eager PyTorch tensors with mutable gradient fields. Backpropagation goes through transformations such as `value_and_grad`.

Correct mental model:

MLX program = build graph $\rightarrow$ transform graph $\rightarrow$ evaluate requested values.

Misconception: Local Graph Correctness Guarantees Distributed Correctness

Each rank may have a locally valid graph, but the distributed program can still deadlock if send and receive orderings do not match globally.

Correct mental model:

Distributed correctness = local graph correctness + global communication protocol correctness.

Misconception: On-Device Means One Device

The lecture's on-device vision includes clusters of local consumer devices. The data remains on-premise, but computation may be distributed across multiple machines.

Practical Takeaways

Engineering Takeaways

- Apple Silicon is attractive for large local inference because of unified memory capacity.
- For very large models, memory capacity and cost per gigabyte can dominate peak FLOPs.
- Decode is usually memory-bound; prefill is more compute-heavy.
- Pipeline parallelism is easier to implement and has fewer communication events.
- Tensor parallelism can improve throughput only when interconnect latency is low.
- Thunderbolt topology matters because links are point-to-point.
- RDMA over Thunderbolt changes the feasibility of tensor parallelism.
- MLX lazy execution allows elegant prefill-only evaluation but requires care in distributed settings.
- LoRA is the practical route for training or fine-tuning huge models on limited-memory clusters.
- Debugging distributed lazy graphs requires explicit ordering and careful protocol design.

Design Heuristics

Use pipeline parallelism when:

- the model does not fit on one device,
- communication latency is high,
- throughput is less important than feasibility,
- the topology is a simple line or ring.

Use tensor parallelism when:

- the model already fits or can be sharded efficiently,
- low-latency communication is available,
- simultaneous compute across devices is needed,
- the communication library supports efficient collectives.

Use heterogeneous prefill/decode when:

- one device is better for compute-heavy prefill,
- another device is better for memory-bound decode,
- KV-cache transfer can be overlapped with compute,
- network bandwidth does not dominate latency.

Use LoRA training when:

- full fine-tuning does not fit,
- only task adaptation is needed,
- the base model can remain frozen,
- optimizer state must remain small.

Project or Experiment Ideas

Single-Device MLX Experiments

1. Benchmark eager-style MLX evaluation versus `mx.compile` on simple MLP blocks.
2. Measure the effect of evaluating logits versus evaluating only the KV cache during prefill.
3. Compare quantized model decode throughput for 4-bit and 8-bit models.
4. Measure memory usage as context length increases and verify the linear KV-cache formula.

Distributed Inference Experiments

1. Implement two-stage pipeline parallel inference for a toy transformer.
2. Measure pipeline bubble effects with one request versus multiple concurrent requests.
3. Simulate tensor parallel communication cost using:

$$T_{\text{comm}} = 2L \left(\tau + \frac{A}{B} \right).$$

4. Compare Ethernet and Thunderbolt latency and bandwidth for activation transfer.
5. Create a physical ring topology and run a ring all-reduce benchmark.

Heterogeneous Prefill and Decode Experiments

1. Benchmark prefill and decode separately on two different devices.
2. Compute arithmetic intensity estimates for each phase.
3. Transfer KV cache between devices and measure whether overlap hides transfer time.
4. Plot roofline curves for each device and mark prefill and decode points.

Training Experiments

1. Implement LoRA fine-tuning for a small transformer in MLX.
2. Reproduce the auxiliary loss trick:

$$\hat{\ell} = \langle y, \tilde{y} \rangle.$$

3. Build a two-rank pipeline-parallel training example.
4. Intentionally break send/receive ordering and observe deadlock behavior.
5. Add artificial dependencies to enforce correct ordering.

Final Mental Model

Distributed ML on consumer devices is the art of matching the workload to a very different hardware regime. Apple Silicon provides enormous unified memory and attractive power characteristics, but much lower peak FLOPs than datacenter GPUs. This makes it compelling for large local inference, especially memory-bound decode, but less natural for full training.

The central performance equation is:

$$T_{\text{token}} = T_{\text{compute}} + T_{\text{memory}} + T_{\text{communication}} + T_{\text{synchronization}}.$$

For a single Apple device, the main question is whether the model and KV cache fit:

$$M_{\text{weights}} + M_{\text{KV}} + M_{\text{overhead}} \leq M_{\text{unified memory}}.$$

For multiple devices, the main question is whether the communication pattern is compatible with the interconnect:

$$T_{\text{comm}} = N_{\text{messages}} \left(\tau + \frac{S}{B} \right).$$

Pipeline parallelism minimizes message count but leaves devices idle. Tensor parallelism improves simultaneous compute but requires frequent low-latency communication. RDMA over Thunderbolt can shift tensor parallelism from impractical to plausible. Heterogeneous prefill/decode goes one step further by assigning compute-heavy prefill and memory-bound decode to different devices.

MLX adds another layer of interest: lazy execution makes local optimization elegant, but distributed lazy graphs require explicit ordering. The fake-loss trick for pipeline backpropagation captures the spirit of the whole lecture: when the framework does not expose exactly the primitive you need, you can often recover it by understanding the math and the execution model deeply.

The final mental model is this: consumer-device distributed ML is not about pretending a pile of Macs is an H100 cluster. It is about exploiting cheap memory, local privacy, physical topology, lazy graph execution, and careful communication design to make large models run in places where datacenter assumptions do not apply.